

1. Write a Python script using sqlite3 to connect to inventory.db and create a cursor for executing SQL commands.

```
import sqlite3

conn = sqlite3.connect("inventory.db")

cursor = conn.cursor()

print("Database connection successful.")

conn.close()
```

Output:

```
In [1]: runfile('D:/TyBsc39/TY/untitled0.py', wdir='D:/TyBsc39/TY')
Database connection successful.
```

2. Write a Python script using sqlite3 to create the products table with id, name, quantity, and price fields, then commit the changes and close the connection.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    price REAL NOT NULL
)
""")

conn.commit()

print("Products table created successfully.")

conn.close()
```

Output:

```
In [2]: runfile('D:/TyBsc39/TY/untitled1.py', wdir='D:/TyBsc39/TY')
Products table created successfully.
```

3. Write a Python script using sqlite3 to insert single and multiple products into the products table using `execute()` and `executemany()`, then display the affected rows and last inserted ID.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

# Insert single product
cursor.execute(
    "INSERT INTO products (name, quantity, price) VALUES (?, ?, ?)",
    ("Laptop", 50, 1200.50)
)

print("Single product inserted.")
print("Rows affected:", cursor.rowcount)
print("Last inserted ID:", cursor.lastrowid)

# Insert multiple products
products = [
    ("Mouse", 100, 25.00),
    ("Keyboard", 75, 75.00)
]

cursor.executemany(
    "INSERT INTO products (name, quantity, price) VALUES (?, ?, ?)",
    products
)

print("Multiple products inserted.")
print("Rows affected:", cursor.rowcount)
print("Last inserted ID:", cursor.lastrowid)

conn.commit()

conn.close()
```

Output:

```
In [3]: runfile('D:/TyBsc39/TY/Assign4 Q3.py', wdir='D:/TyBsc39/TY')
Single product inserted.
Rows affected: 1
Last inserted ID: 1
Multiple products inserted.
Rows affected: 2
Last inserted ID: 1
```

4. Write a Python script using sqlite3 to retrieve all products, find a product by name, and display products with quantity below 60 using SELECT.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM products")

products = cursor.fetchall()

print("----- All Products -----")

for product in products:
    print(
        f"ID: {product[0]}, "
        f"Name: {product[1]}, "
        f"Quantity: {product[2]}, "
        f"Price: {product[3]}"
    )

cursor.execute(
    "SELECT * FROM products WHERE name = ?",
    ("Laptop",)
)

product = cursor.fetchone()

print("\n----- Specific Product -----")

if product:
    print(
        f"ID: {product[0]}, "
        f"Name: {product[1]}, "
        f"Quantity: {product[2]}, "
        f"Price: {product[3]}"
    )
else:
    print("Product not found.")

cursor.execute(
    "SELECT * FROM products WHERE quantity < ?",
    (60,)
)

low_quantity = cursor.fetchall()

print("\n----- Products with Quantity Less Than 60 -----")
```

```
for product in low_quantity:
    print(
        f"ID: {product[0]}, "
        f"Name: {product[1]}, "
        f"Quantity: {product[2]}, "
        f"Price: {product[3]}"
    )

conn.close()
```

Output:

```
In [4]: runfile('D:/TyBsc39/TY/Assign4 Q4.py', wdir='D:/TyBsc39/TY')
----- All Products -----
ID: 1, Name: Laptop, Quantity: 50, Price: 1200.5
ID: 2, Name: Mouse, Quantity: 100, Price: 25.0
ID: 3, Name: Keyboard, Quantity: 75, Price: 75.0

----- Specific Product -----
ID: 1, Name: Laptop, Quantity: 50, Price: 1200.5

----- Products with Quantity Less Than 60 -----
ID: 1, Name: Laptop, Quantity: 50, Price: 1200.5
```

5. Write a Python script using sqlite3 to update the quantity and price of products in the products table and display the affected rows.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

cursor.execute(
    "UPDATE products SET quantity = ? WHERE name = ?",
    (45, "Laptop")
)

print("Laptop quantity updated.")
print("Rows affected:", cursor.rowcount)

cursor.execute(
    "UPDATE products SET price = ? WHERE name = ?",
    (80.00, "Keyboard")
)

print("Keyboard price updated.")
print("Rows affected:", cursor.rowcount)

conn.commit()

conn.close()
```

Output:

```
In [5]: runfile('D:/TyBsc39/TY/Assign4 Q5.py', wdir='D:/TyBsc39/TY')
Laptop quantity updated.
Rows affected: 1
Keyboard price updated.
Rows affected: 1
```

6. Write a Python script using sqlite3 to delete a product from the products table by its ID and display the affected rows.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

cursor.execute(
    "DELETE FROM products WHERE id = ?",
    (3,)
)

print("Product deleted.")
print("Rows deleted:", cursor.rowcount)

conn.commit()

conn.close()
```

Output:

```
In [6]: runfile('D:/TyBsc39/TY/Assign4 Q6.py', wdir='D:/TyBsc39/TY')
Product deleted.
Rows deleted: 1
```

7. Write a Python script using sqlite3 to demonstrate commit() and rollback() by inserting products and verifying the changes.

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

cursor.execute(
    "INSERT INTO products (name, quantity, price) VALUES (?, ?, ?)",
    ("Tablet", 20, 300.00)
)

print("Tablet inserted.")

conn.commit()

print("Commit completed.")

# Display products
cursor.execute("SELECT * FROM products")
products = cursor.fetchall()

print("\n----- Products After Commit -----")

for product in products:
    print(
        f"ID: {product[0]}, "
        f"Name: {product[1]}, "
        f"Quantity: {product[2]}, "
        f"Price: {product[3]}"
    )

cursor.execute(
    "INSERT INTO products (name, quantity, price) VALUES (?, ?, ?)",
    ("Webcam", 50, 45.00)
)

print("\nWebcam inserted.")

conn.rollback()

print("Rollback completed.")

cursor.execute("SELECT * FROM products")
products = cursor.fetchall()

print("\n----- Products After Rollback -----")

for product in products:
    print(
```

```
f"ID: {product[0]}, "  
f"Name: {product[1]}, "  
f"Quantity: {product[2]}, "  
f"Price: {product[3]}"  
)  
  
conn.close()
```

Output:

```
In [7]: runfile('D:/TyBsc39/TY/Assign4 Q7.py', wdir='D:/TyBsc39/TY')  
Tablet inserted.  
Commit completed.  
  
----- Products After Commit -----  
ID: 1, Name: Laptop, Quantity: 45, Price: 1200.5  
ID: 2, Name: Mouse, Quantity: 100, Price: 25.0  
ID: 4, Name: Tablet, Quantity: 20, Price: 300.0  
  
Webcam inserted.  
Rollback completed.  
  
----- Products After Rollback -----  
ID: 1, Name: Laptop, Quantity: 45, Price: 1200.5  
ID: 2, Name: Mouse, Quantity: 100, Price: 25.0  
ID: 4, Name: Tablet, Quantity: 20, Price: 300.0
```

8. Write a Python script using sqlite3 to back up the inventory.db database into an inventory_backup.sql file using iterdump().

```
import sqlite3

conn = sqlite3.connect("inventory.db")

with open("inventory_backup.sql", "w") as file:
    for line in conn.iterdump():
        file.write(line + "\n")

print("Database backup created successfully.")

conn.close()
```

Output:

```
In [8]: runfile('D:/TyBsc39/TY/Assign4 Q8.py', wdir='D:/TyBsc39/TY')
Database backup created successfully.
```

9. Write a Python script using sqlite3 and csv to export all products from the products table to products_export.csv with column headers.

```
import sqlite3
import csv

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM products")

products = cursor.fetchall()

column_names = [description[0] for description in cursor.description]

with open("products_export.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(column_names)

    writer.writerows(products)

print(
    f"{len(products)} products exported successfully to products_export.csv"
)

conn.close()
```

Output:

```
In [9]: runfile('D:/TyBsc39/TY/Assign4 Q9.py', wdir='D:/TyBsc39/TY')
3 products exported successfully to products_export.csv
```